1. Investigate
Columns for investigation: CARRIER, CARRIER_NAME, MANUFACTURE_YEAR, NUMBER_OF_SEATS, CAPACITY_IN_POUNDS, and AIRLINE_ID.

For column CARRIER, the only values that are missing are those of with CARRIER_NAME North American Airlines, with 49 missing. All missing values can be imputed with North American Airlines' CARRIER value.
Code:

```
# Fill carrier with Carrier_name. I googled Nort American Airline's CARRIER code.
    carrier_mask = (
        df["CARRIER_NAME"].eq("North American Airlines") &
        df["CARRIER"].isna()
    )
    df.loc[carrier_mask, "CARRIER"] = "NAA"
```

For column CARRIER_NAME, Carrier OH didn't fill in CARRIER_NAME or UNIQUE_CARRIER_NAME 97 times.
For column CARRIER_NAME, Carrier L4 didn't fill in CARRIER_NAME or UNIQUE_CARRIER_NAME 8 times. Missing values with Carrier OH will be imputed with their CARRIER_NAME value, and missing values with Carrier L4, will be imputed with their CARRIER_NAME value.
Code:

```
# Filling Carrier_name by Carriers OH and L4.
    oh_carrier_name_mask = (df["CARRIER"] == "OH") & (df["CARRIER_NAME"].isna())
    oh_name = df.loc[df["CARRIER"] == "OH", "CARRIER_NAME"].dropna().mode()
    if len(oh_name) > 0:
        df.loc[oh_carrier_name_mask, "CARRIER_NAME"] = oh_name.iloc[0]

    l4_carrier_name_mask = (df["CARRIER"] == "L4") & (df["CARRIER_NAME"].isna())
    l4_name = df.loc[df["CARRIER"] == "L4", "CARRIER_NAME"].dropna().mode()
    if len(l4_name) > 0:
        df.loc[l4_carrier_name_mask, "CARRIER_NAME"] = l4_name.iloc[0]
```

The next three columns all use this median impute helper function:

```
def median_impute_helper(df, column, group_order):
        for col in group_order:
            df[column] = df[column].fillna(
                df.groupby(col)[column].transform("median")
            )

        df[column] = df[column].fillna(df[column].median())
        return df
```

For column MANUFACTURE_YEAR, only three rows are missing, two from carrier Endeavor Air Inc. and one from carrier Atlas Air Inc. These values can be filled using the median value of those with the same model.
Code:

```
df = median_impute_helper(
        df,
        column="MANUFACTURE_YEAR",
        group_order=["TAIL_NUMBER", "MODEL", "MANUFACTURER"]
    )
```

For column CAPACITY_IN_POUNDS, it's likely Missing at Random (MAR). Certain models, aircraft types and manufacturers are always missing the capacity in pounds. Additionally, certain carriers are highly likely to be missing this column, such as “TEM Enterprises dba Avelo Airlines” always missing and “Breeze Aviation Group DBA” missing 82% of the time. I would first impute using the median value from aircraft of the same model, then same type of aircraft, then the median of all aircraft.
Code:

```
df = median_impute_helper(
        df,
        column="CAPACITY_IN_POUNDS",
        group_order=["TAIL_NUMBER", "MODEL", "AIRCRAFT_TYPE", "MANUFACTURER"]
    )
```

For column NUMBER_OF_SEATS, it's likely Missing At Random (MAR). Certain models, aircraft types, carriers, and tail numbers are always missing the number of seats. Additionally, the mean capacity in pounds where the number of seats is missing vs filled is 167576.49 and 58619.28 pounds, indicating that larger aircraft are more likely to be missing the number of seats. I would first impute using the median value from aircraft of the same model, then same type of aircraft, then the median of all aircraft.
Code:

```
df = median_impute_helper(
        df,
        column="NUMBER_OF_SEATS",
        group_order=["TAIL_NUMBER", "MODEL", "AIRCRAFT_TYPE", "MANUFACTURER"]
    )
```

2. Inspect

Inspect the columns MANUFACTURER, MODEL, AIRCRAFT_STATUS, and OPERATING_STATUS. Decide, for each column, if transformation or standardization of data are required. Give your reasoning and code if you decide to transform the data

For column MANUFACTURER, I looked at the top 20 value counts, and immediately noticed that many companies are listed multiple times with different spellings or capitalization. The table below lists alternative names. I standardized manufacturer name capitalizations to be all uppercase. I continued finding name duplicates until all categories with more than 1000 entries were merged correctly.

Manufacturer	Alternative names found
BOEING	THEBOEINGCO, Boeing, BoeingCo, BOEINGCOMPANY, THEBOEINGCOMPANY, TheBoeingCompany
Embraer	EMBRAER
Bombardier	BOMBARDIER
AIRBUS	AirbusIndustries, Airbus, AirbusIndustrie
Many other examples…	

For column MODEL, I first noticed that the 737 has many different models and some are designated as passenger only. The passenger designation resulted in many of the same model being listed differently. I was able to combine roughly 2500 entries across 6 slightly different model names into one model name. I noticed there were many names that had a few characters on the end that differentiated them, such as LR (meaning long range0, but I didn't want to erase significant information by overaggressively combining model names. I looked at the most frequently occurring models, then looked at lists containing keywords that the most significant model had, for example:

```
df[df["MODEL"].str.contains("737-700", na=False)]["MODEL"].value_counts().head(20)
```

to find model names for 737-700 that I then combined into 737-700PASSENGERONLY.

My main focus was on congregating various spellings of the top 10 most models. By the end of my processing, the top 10 had 33,188 occurrences, in comparison to 21,102 occurrences before name standardization.

For column AIRCRAFT_STATUS, I completed feature engineering before this step, so I had already completed data standardization, reasoning here. Since the only possible letters were A, B, or O, I assumed L was a typo for O due to their proximity on the keyboard, and replaced all instances of L with O.

Code:

```
df["AIRCRAFT_STATUS"] = df["AIRCRAFT_STATUS"].replace({'A': 'a', 'B': 'b', 'o': 'O', 'L': 'O'})
```

Same for OPERATING_STATUS, I completed data standardization in feature engineering. I also had to remove one empty string value.
Code:

```
df["OPERATING_STATUS"] = df["OPERATING_STATUS"].replace({'y':'Y'})
df = df[df["OPERATING_STATUS"] != ""]
```

3. Drop N/A

Before dropping N/A values, there are 132,313 rows. After dropping N/A values, there are 101,276 rows.

4. Transformation

For column NUMBER_OF_SEATS, before Box-Cox, the skew was -0.29, which is relatively close to normal. Something I noticed was that there was a large number of 0 values for seat numbers and there were also clusters, such as a large number of aircraft with 50 seats. This is likely due to various aircraft models being popular and having a standardized number of seats. After Box-Cox, the skew's absolute value increased to -0.68. Additionally, I don't think it's a good representation of the distribution of number of seats, since number of seats isn't a normal distribution, even though the original distribution only has a skew of -0.29. The number of seats has several distinct values that it clusters at, such as the natural large number of 0 seats due to the original data.

For column CAPACITY_IN_POUNDS, before Box-Cox, the skew was 3.78, indicating that it is very right-skewed. This means that the mean will be higher than the median. Box-Cox was quite useful for this column, reducing the skew to only 0.21, and creating a relatively normal distribution from a previously very skewed distribution. This prevents a few very large aircraft from influencing the distribution of the column more than they should.

5. Feature Engineering

The majority of aircraft in all size categories are operating, with the medium category having the highest proportion of non-operating aircraft. Additionally, there is some non-standardized data entry, such as entering aircraft operation status as “Y” vs ‘y’. I did standardize the data before plotting, but there was a smaller number of non-standard entry than aircraft status, so there wasn’t much of a visual difference.

The meaning of the data in Aircraft Status seems to be the type of lease, based on this website: https://esubmit.rita.dot.gov/F41F-B-43.aspx. Specifically, a = Capital Lease, b = Operating Lease, and O = Owned. I found that there was a lot of unstandardized data entry, specifically capitalization when entering this data, which made it difficult to understand at first. Thus, I standardized the capitalization for the types of leases or ownership.

```
AIRCRAFT_STATUS
O    79487
b    30852
B    12699
a     7804
A     1330
L      122
o       19
Name: count, dtype: int64
```

Above is the count of each category in aircraft_status before I standardized the capitalization. After I standardized capitalization, it was much simpler to see the overall trends. In all size categories but medium, the most common aircraft status is owned, but in Medium the most common status is operating lease. It’s likely that larger and smaller aircraft are used for routine operations such as commercial aircraft or personal use, while medium aircraft are leased out for less routine usage. The least common status for all aircraft sizes is capital lease.

Below are the value proportions by size for AIRCRAFT_STATUS and OPERATING_STATUS.

```
SIZE    AIRCRAFT_STATUS
LARGE   O                 59.65
        b                 27.76
        a                 12.43
        L                  0.16
MEDIUM  b                 57.46
        O                 39.49
        a                  3.05
SMALL   O                 72.11
        b                 24.25
        a                  3.64
XLARGE  O                 68.54
        b                 23.15
        a                  8.13
        L                  0.17
Name: proportion, dtype: float64
```

```
SIZE    OPERATING_STATUS
LARGE   Y                 95.66
        N                  4.34
MEDIUM  Y                 93.37
        N                  6.63
                           0.00
SMALL   Y                 95.95
        N                  4.05
XLARGE  Y                 97.37
        N                  2.63
Name: proportion, dtype: float64
```

6. Modeling

```
Linear Regression predicting NUMBER_OF_SEATS
Train RMSE: 12.4191
Test  RMSE: 12.4737

Random Forest predicting NUMBER_OF_SEATS
Train RMSE: 0.7074
Test  RMSE: 1.0332

Linear Regression predicting CAPACITY_IN_POUNDS
Train RMSE: 24.8251
Test  RMSE: 24.8450

Random Forest predicting CAPACITY_IN_POUNDS
Train RMSE: 1.4737
Test  RMSE: 2.6103
```

The Random Forest model performed better for both the number of seats and capacity in pounds. I hypothesized that linear regression wouldn't perform well since the relationship for number of seats and capacity in pounds is not linear with other factors. Additionally, number of seats is highly clustered. However, the extremely low Train RMSE for both Random Forests suggests that the model is likely overfitted. On the other hand, since the RMSE for train and test of the Linear Regression models is very close to each other (difference of ~0.06), this suggests tha the the Linear Regression models are not overfit.

AI Usage: None.